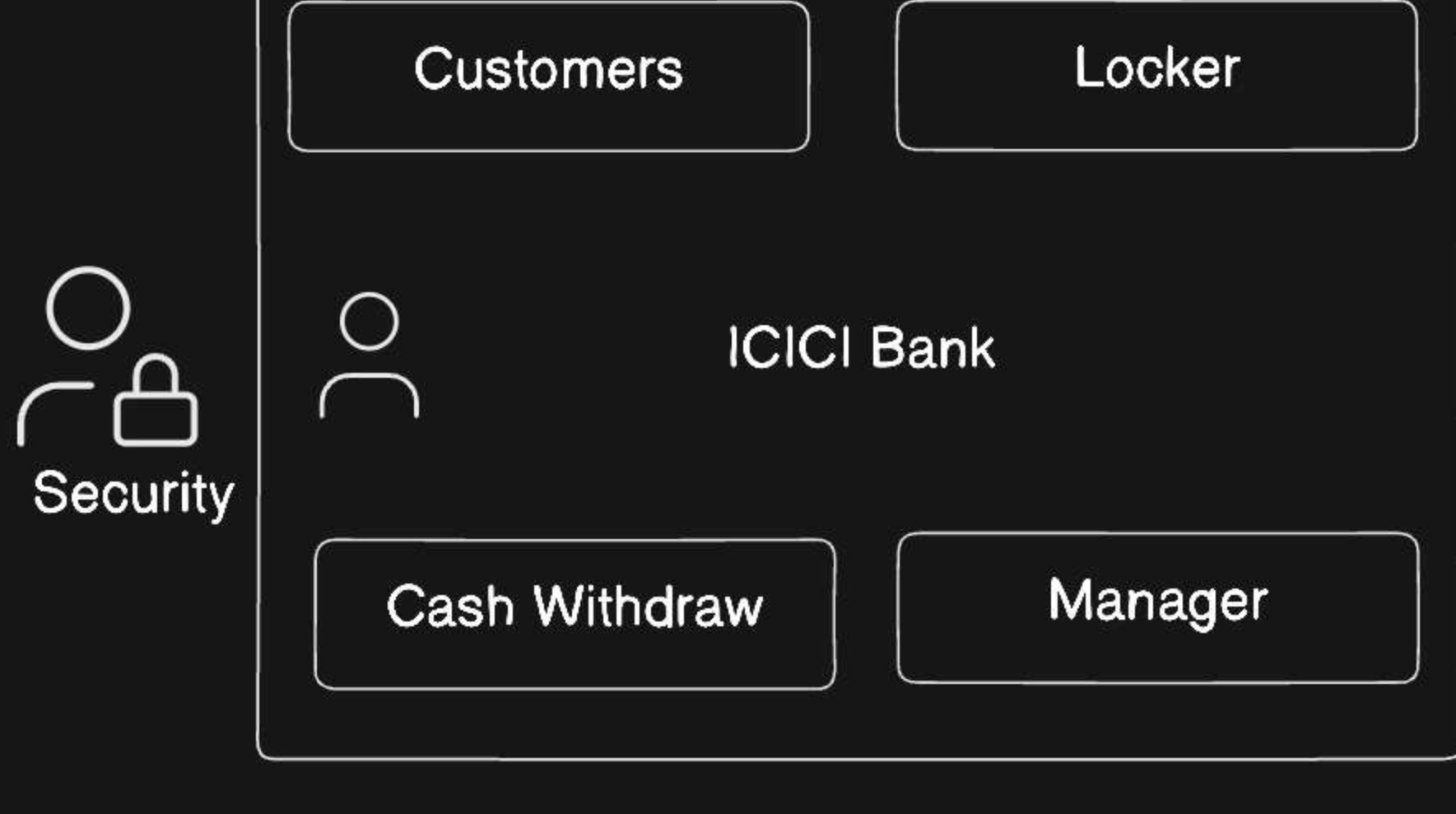


1. Authorization
2. RBAC (Role-Based Access Control)
3. ABAC (Attribute-Based Access Control)

Authorization

Authorization is the process of deciding what an authenticated user is allowed to access or perform.

Login -> Authenticate -> Authorization



Authentication: Security gurd checks your ID Card or Gov Id proof. If those are valid then security gaurd allow you to enter in Bank

Authorization: Based on your role, you can enter in respective departmets.

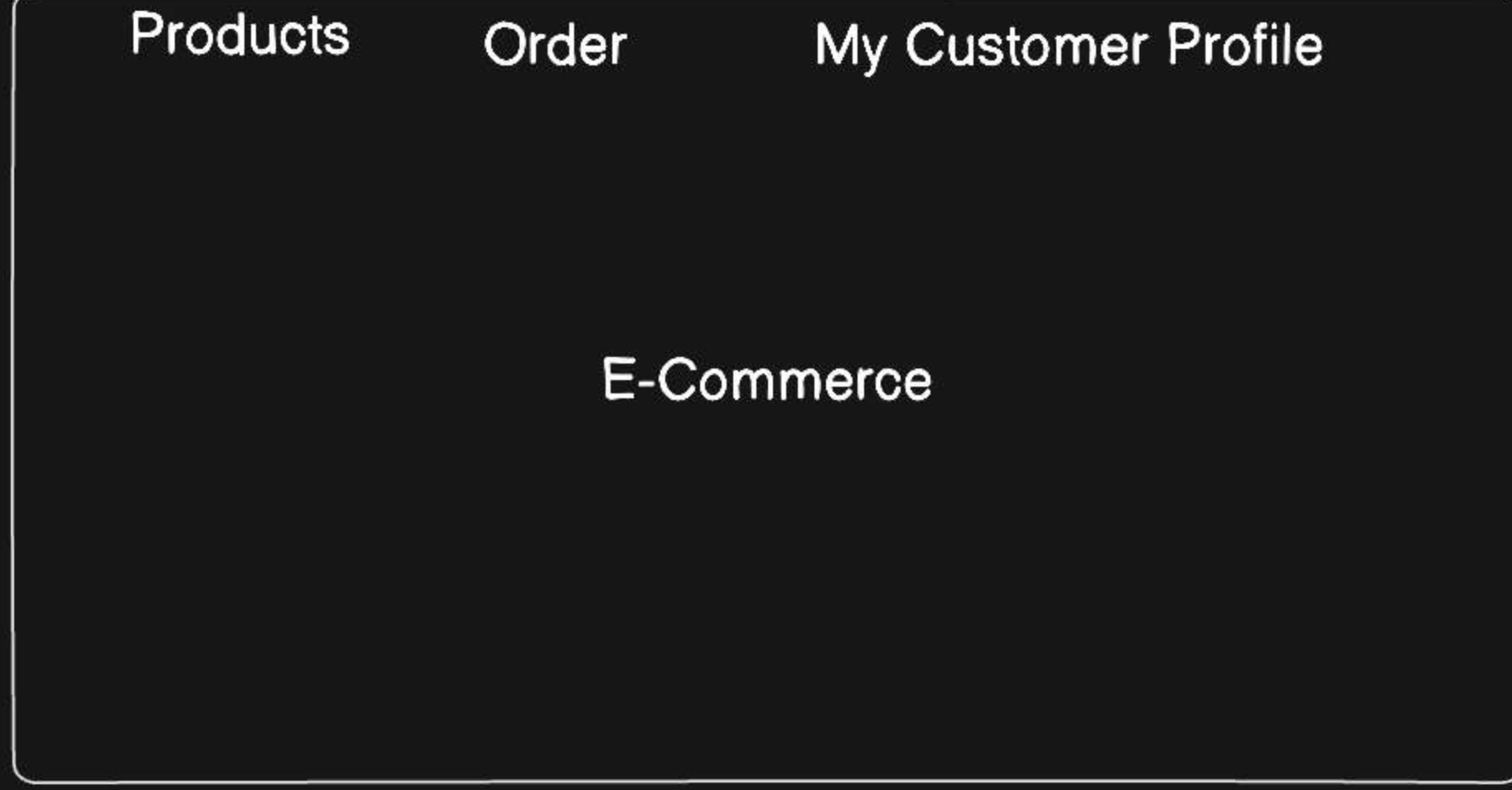
Authorization Techniuge:

1. RBAC
2. ABAC

Role-Based Access Control (RBAC)

RBAC is an authorization model where permissions are assigned to roles, and users inherit permissions through their assigned roles.

Example: E-Commerce



1. Admin
2. Seller
3. Customer

Admin - Mange Products, Mangae Orders, Manage Customers, Manage Sellers, View All

Seller - Mange Own Products, Manage Own Product Orders,

Customer - View Products, Manage Own Orders

```
// RBAC Check: Simple string match
function canAccessEndpoint(user, requiredRole) {
  return user.roles.includes(requiredRole);
}

// Usage inside a router/controller
if (canAccessEndpoint(currentUser, 'Seller')) {
  renderSellerDashboard();
} else {
  throw new Error("403 Unauthorized: Requires Seller Role");
}
```

Advantages:

- Easy to implement
- Easy to understand

Disadvantages:

- Too many roles as the system grows
- Difficulut to handle special cases

Attribute-Based Access Control (ABAC)

ABAC is an authorization model where access decisions are based on attributes of the user, resource, action, and environment.

- User (Subject) - Age, Department, Clerance level etc
- Resource (Object) - File Type, Money, Owner, created date etc
- Action (Operation) - Read, Update, Delete, Approve etc
- Environment (Context) - Geo location, current time, IP Address, device type etc

Example: Enterprise Financial App

1. A user can Approve a Financial Transfer ONLY IF they belong to the "Finance" department
2. The transfer amount is less than Rs 50000
3. It is between 9:00 AM and 5:00 PM
4. They are connecting from a company owned laptop.

```
function evaluateFinancialTransferPolicy(subject, resource, action, environment) {
  // Condition 1: Must belong to the "Finance" department
  if (subject.department !== 'Finance') {
    return { authorized: false, reason: "User does not belong to the Finance department." };
  }

  // Condition 2: The transfer amount must be less than Rs 50,000
  if (action === 'APPROVE' && resource.amount >= 50000) {
    return { authorized: false, reason: "Transfer amount meets or exceeds the Rs 50,000 threshold." };
  }

  // Condition 3: Must be between 9:00 AM and 5:00 PM
  const currentHour = environment.currentTime.getHours();
  if (currentHour < 9 || currentHour >= 17) {
    return { authorized: false, reason: "Action attempted outside authorized business hours (9:00 AM - 5:00 PM)." };
  }

  // Condition 4: Must be connecting from a company-owned laptop
  if (!environment.isCompanyOwnedLaptop) {
    return { authorized: false, reason: "Connection source is not a verified company-owned laptop." };
  }

  // All conditions passed successfully
  return { authorized: true, reason: "Access granted." };
}
```

```
const user = { id: "EMP101", department: "Finance" };
const transfer = { amount: 35000, currency: "INR" };
const env = { currentTime: new Date("2026-07-20T14:30:00"), isCompanyOwnedLaptop: true }; // 2:30 PM

const result = evaluateFinancialTransferPolicy(user, transfer, 'APPROVE', env);
console.log(result.authorized); // Output: true
```

Advantages:

- Very Flexible
- Fewer roles
- Excelent for enterprise application

Disadvantages:

- Complex implementaion
- Testing is difficlut
- Hard to understand

